Text Mining in R – Sentiment Analysis

Gosse Bouma, Defne Abur

February, 2025

Goal

The goal of this assignment is:

- to get some experience with converting text files to a data frame (and then tokenizing it)
- to experiment with some of the techniques in chapter 2 of the Text Mining book

Amazon book review dataset

Amazon provides a large corpus of reviews: https://jmcauley.ucsd.edu/data/amazon/ In this assignment we will focus on book reviews.

Book reviews typically consist of text and a rating, typically using the 5 star system. The sample I have created contains negative reviews (1 or 2 stars) and positive reviews (4 or 5 stars); i.e., neutral reviews with 3 stars have been left out.

Datasets like this are often used for sentiment polarity classification; i.e., to predict on the basis of the text whether a review is positive or negative. Here we want to use it as **two corpora**, and study the sentiment scores in both subcorpora. Of course, we expect that the positive reviews contain more positive words than the negative reviews, and vice versa.

For this assignment, I made 10 subsets of positive reviews (each file containing a sample of 500 reviews) and vice versa for negative reviews. You will be working with the positive and negative sample that corresponds to the last number of your student number (i.e. student s123456 will work with pos6.txt and neg6.txt). All data can be found on Google drive

My Snumber is s5788668, so I will be working with pos8.txt and neg8.txt.

Reading and tokenizing textual data

The readr package provides a function `read_lines()` that can be used to read arbitrary text and store the result as a vector of strings. Use this to read in the data.

Next, create a data frame for the data, and tokenize it as you did in the first R assignment.

```
pos <- read_lines(here("data","raw", "sentimentAnalysis", "pos8.txt"))
neg <- read_lines(here("data","raw", "sentimentAnalysis", "neg8.txt"))
sentiment <- data_frame(
  review = c(pos, neg),
  sentiment = c(rep("pos", length(pos)),rep("neg", length(neg)))) %>%
  unnest_tokens(word, review)
```

```r
#check the head and tail of the data
head(sentiment)
```

```
## # A tibble: 6 x 2
##   sentiment word
##   <chr>     <chr>
## 1 pos       as
## 2 pos       usual
## 3 pos       elizabeth
## 4 pos       lowell
## 5 pos       thrills
## 6 pos       all
```

```r
tail(sentiment)
```

```
## # A tibble: 6 x 2
##   sentiment word
##   <chr>     <chr>
## 1 neg       her
## 2 neg       book
## 3 neg       i
## 4 neg       smelled
## 5 neg       a
## 6 neg       rat
```

Sentiment Analysis

Analyze the text using the *afinn* polarity lexicons provided by tidytext (see Chapter 2 of the Text Mining book). This lexicon scores words on a scale from -5 (negative) to 5 (positive) in a column named "value"; whenever this assignment talks about a sentiment score, it refers to a score made of the numbers in this "value" column. To get this lexicon, you need to manually install a separate package and agree to the license of AFINN (you only need to do this once). Run the following code from the R console, and answer yes to the license conditions:

```
install.packages("textdata")
library(tidytext)
get_sentiments("afinn")
```

Once this works, complete the rest of this notebook. Provide code to answer the following questions:

- What is the overall sentiment score for the positive reviews and for the negative reviews? An overall sentiment score is the sum of all the values assigned by the AFINN lexicon for the words in those reviews.

```r
# Preprosessing: remove stop words
data("stop_words")
sentiment_clean <- anti_join(sentiment, stop_words)
```

```r
# Question 1: the overall sentiment score for pos/neg reviews
sentiment_clean %>%
  inner_join(get_sentiments("afinn")) %>% # OOV could be an issue
  group_by(sentiment) %>%
  summarise(sentiment_score = sum(value))
```

```
## # A tibble: 2 x 2
##   sentiment sentiment_score
##   <chr>               <dbl>
## 1 neg                 -1728
## 2 pos                  2344
```

- Which words with a sentiment score of 4 or 5 are most frequent in the positive reviews? And in the negative reviews? Give the top 5 words returned by your solution as examples.

```
# Question 2: the most frequent words with a sentiment score of 4 or 5
sentiment_clean %>%
  inner_join(get_sentiments("afinn")) %>%
  filter(value >= 4 | value <= -4) %>%
  group_by(sentiment) %>% # grouping should before count so the pos/neg will be sorted separately
  count(word, sentiment, sort = TRUE) %>%
  #group_by(sentiment) %>% -- don't do this
  top_n(5, n)
```

```
## # A tibble: 10 x 3
## # Groups:   sentiment [2]
##    sentiment word          n
##    <chr>     <chr>     <int>
##  1 pos       wonderful    49
##  2 pos       fun          46
##  3 neg       funny        31
##  4 pos       funny        30
##  5 pos       amazing      17
##  6 neg       fun          16
##  7 pos       fantastic    14
##  8 neg       wonderful    13
##  9 neg       amazing       8
## 10 neg       hell          8
```

- Which words with a sentiment score of -4 or -5 are most frequent in the positive reviews? And in the negative reviews? (Be prepared for some strong language.) Give the top 5 words returned by your solution as examples.

Note that R provides functions for summing all values in a given column as well as for selecting subsets of a given data frame.

```
# Question 3: the most frequent words with a sentiment score of -4 or -5
# The logic is basically the same, except we removed the "or" in the filter
# in that we only focus on the polarised neg words
sentiment_clean %>%
  inner_join(get_sentiments("afinn")) %>%
  filter(value <= -4) %>%
  group_by(sentiment) %>%
  count(word, sentiment, value, sort = TRUE) %>%
  top_n(5, n)
```

```
## # A tibble: 11 x 4
## # Groups:   sentiment [2]
```

```
##    sentiment word     value     n
##    <chr>     <chr>    <dbl> <int>
##  1 neg       hell        -4     8
##  2 neg       fraud       -4     6
##  3 neg       damn        -4     5
##  4 neg       rape        -4     4
##  5 neg       ass         -4     3
##  6 pos       tortured    -4     2
##  7 pos       bitch       -5     1
##  8 pos       damn        -4     1
##  9 pos       hell        -4     1
## 10 pos       rape        -4     1
## 11 pos       torture     -4     1
```

NB: some of the words in the sentiment words are very frequent stop words such as "like". It may be a good idea to filter out these stop words from the reviews, because otherwise you may get odd results where negative reviews get an overall positive score.

Which words contribute most?

If your positive reviews have a sentiment score of 1000, you might wonder which words actually contribute most to this score. The contribution of a word to the sentiment score is the product of its sentiment value in the lexicon (between -5 and 5) and its frequency (n if you used count).

- For the positive reviews, provide code that adds a column *contribution* to a data frame with word counts (as produced by count) and give the 10 words that contribute most to the sentiment score.

```r
# Question 4 - 1: the words that contribute most to the sentiment score
sentiment_clean %>%
  filter(sentiment == "pos") %>%
  inner_join(get_sentiments("afinn")) %>%
  count(value, word) %>%
  mutate(contribution = n * value) %>%
  arrange(desc(contribution)) %>%
  slice_head(n = 10)
```

```
## # A tibble: 10 x 4
##    value word          n contribution
##    <dbl> <chr>     <int>        <dbl>
##  1     3 love        151          453
##  2     4 wonderful    49          196
##  3     4 fun          46          184
##  4     3 loved        61          183
##  5     2 recommend    61          122
##  6     4 funny        30          120
##  7     2 true         54          108
##  8     3 fan          34          102
##  9     3 perfect      33           99
## 10     2 favorite     48           96
```

- For the negative reviews, do the same, but now give the words that contribute most in a negative sense.

```r
# Question 4 - 2: the words that contribute most to the sentiment score
sentiment_clean %>%
  filter(sentiment == "neg") %>%
  inner_join(get_sentiments("afinn")) %>%
  filter(value < 0) %>%  # Add this line to check neg words only
  count(value, word) %>%
  mutate(contribution = n * value) %>%
  #arrange(desc(contribution)) %>% -- remove it to show the most neg words
  slice_head(n = 10)
```

```
## # A tibble: 10 x 4
##    value word             n contribution
##    <dbl> <chr>        <int>        <dbl>
##  1    -5 bastards          1           -5
##  2    -5 bitch             1           -5
##  3    -5 cocksucker        1           -5
##  4    -5 slut              1           -5
##  5    -4 ass               3          -12
##  6    -4 catastrophic      2           -8
##  7    -4 damn              5          -20
##  8    -4 damned            1           -4
##  9    -4 dick              2           -8
## 10    -4 fraud             6          -24
```

Note that `count(value, word)` will create a data frame that contains both the sentiment value of a word and its count. You can use this to compute the contribution of a word to the final score as the product of `n` and `value`. Add this contribution as a new column to the data frame using mutate (or the standard method for doing this in R).

Sorting of a data frame can be done using the `arrange` function from dplyr (or the `order` function from standard R).

Contrasting positive and negative reviews

A more *data-driven* approach to studying the difference between positive and negative reviews is to search for words that are more frequent in one as compared to the other, instead of using a dictionary.

Section 1.5 of the Text Mining in R book illustrates how to contrast three corpora. Here we use the same technique but only on two corpora.

Follow the steps as in section 1.5 to create a 'frequency' data frame; i.e. a data frame containing word counts for both the positive and negative reviews (using `bind_rows`), a column for each 'group' (positive and negative reviews in this case) with the relative frequency of each word (i.e., its count divided by the total number of words in the corpus), and then creating a transformed data frame that has relative frequencies per word spread over two columns (positive and negative).

```r
# Question 5

#I'm not sure if I understand this question correct,
#so I will keep as many intermediate steps as possible for later debugging.

# Step 1 calculate word counts and totals for each corpus
```

```r
# Start by calculating the positive words
pos_words <- sentiment_clean %>%
  filter(sentiment == "pos") %>%
  count(word, name = "pos_count") # Don't use default name, forget immediately

# Do the same to the negtive words
neg_words <- sentiment_clean %>%
  filter(sentiment == "neg") %>%
  count(word, name = "neg_count")

# Calculate total number of words in each corpus
pos_total <- sum(pos_words$pos_count)
neg_total <- sum(neg_words$neg_count)

# Step 2 Put them together and make the dataframe
frequency <- bind_rows(
  # Also add relative freq
  pos_words %>%
    mutate(
      corpus = "positive",
      proportion = pos_count / pos_total
    ) %>%
    select(word, corpus, count = pos_count, proportion),

  neg_words %>%
    mutate(
      corpus = "negative",
      proportion = neg_count / neg_total
    ) %>%
    select(word, corpus, count = neg_count, proportion)

)

# Step 3 Data reshaping
word_proportions <- frequency %>%
  select(word, corpus, proportion) %>%
  pivot_wider( # based on the instruction, the operation should be long -> wide
    names_from = corpus,
    values_from = proportion,
    values_fill = 0 # keep it for now, may cause mistake
  )
```

Some explanation of the `spread` function can be found here. As we only have two 'groups' (positive and negative reviews), there is no need for the gather step applied in section 1.5 but it cannot do harm either.... According to `tidyr` 1.3.1 documentation, `pivot_wider/longer` is the new version of `spread/gather` functions. I follow the latest version to avoid potential issues.

Next, follow the steps in 1.5 used to create a scatter plot contrasting the two corpora.

- What are 5 words typically associated with (i.e., more frequent in) positive reviews? And 5 words typically associated with negative reviews?

You may have to play a bit with the `geom_jitter` parameters (esp. `alpha`) to see more words (instead of just dots). As you are producing only a single scatter plot (and not a set of plots arranged in a single impage)

there also is no need for `facet_wrap`.

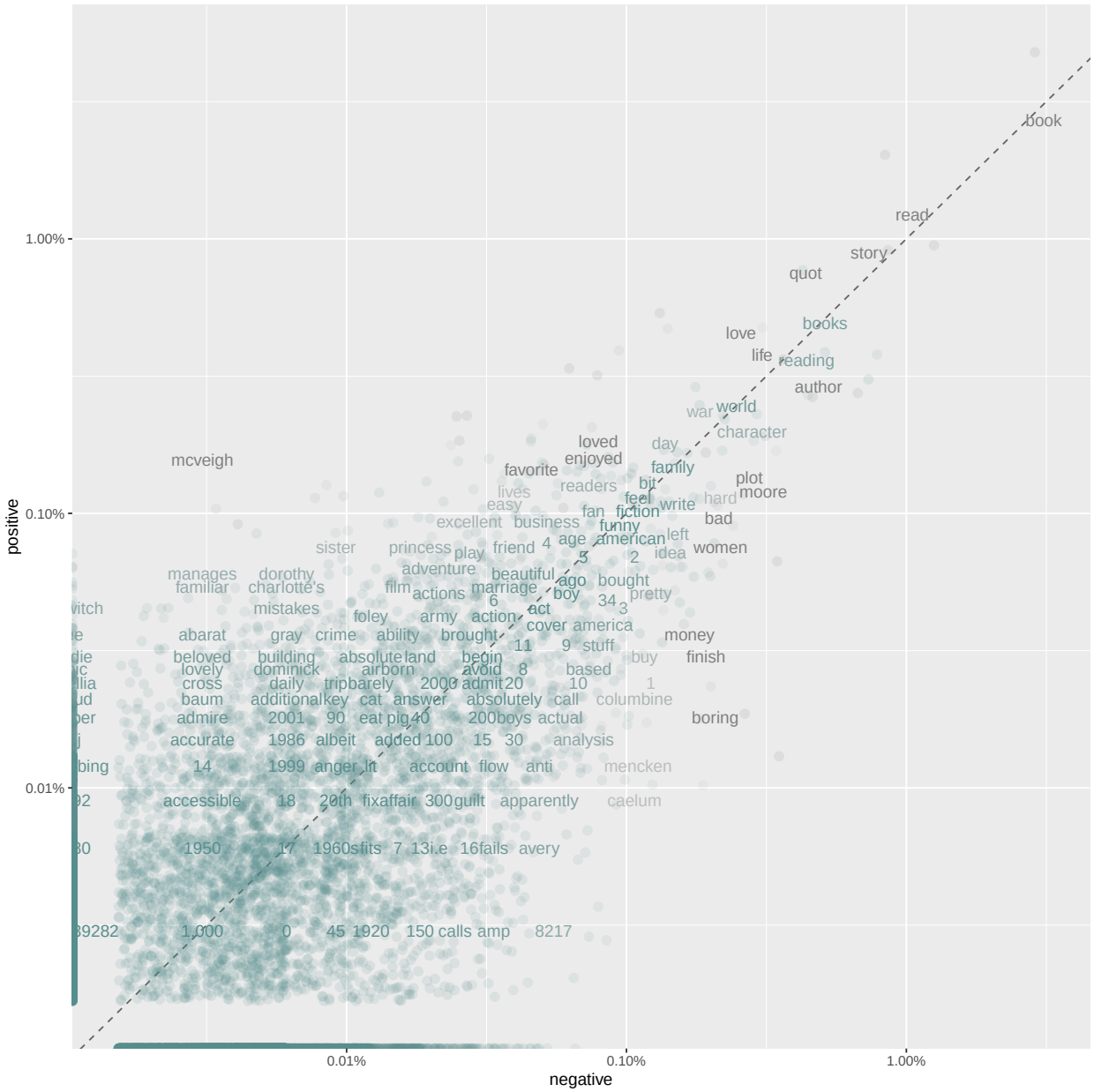
The setting for `fig.width` and `fig.height` in the R code settings below make sure that the image is printed as a large image on my screen. You may want to adjust that for your hardware.

Also, remove the eval=FALSE flag once you have created the data for pos and neg.

```
library(tidyr)
library(ggplot2)
library(scales)

library(tidyr)
library(ggplot2)
library(scales)

ggplot(word_proportions,
       aes(x = negative, y = positive, color = abs(positive - negative))) +
  geom_abline(color = "gray40", lty = 2) +
  geom_jitter(alpha = 0.12, size = 2.5, width = 0.3, height = 0.3) +
  geom_text(aes(label = word), check_overlap = TRUE, vjust = 1.5) +
  scale_x_log10(labels = percent_format()) +
  scale_y_log10(labels = percent_format()) +
  scale_color_gradient(limits = c(0, 0.001), low = "darkslategray4", high = "gray75") +
  theme(legend.position="none") +
  labs(y = "positive", x = "negative")
```



Critical reflection

- Look critically at the sentiment lexicon. Sentiment lexicons are known to be sensitive to the domain (in this case: books). Is this a good lexicon for this domain, or do you run into a lot of words with the wrong sentiment score in the context of book reviews?

```
affin_results <- inner_join(get_sentiments("afinn"), sentiment_clean) %>%
  select(sentiment, word, value)
```

The AFINN results show some clear limitations. One of the obvious issues is how it handles words like "agree" - the lexicon gives it a positive score (+1). However it appears frequently in both positive and negative reviews. This shows that AFINN can't understand the context of words.

- How does the data-driven approach differ from the sentiment lexicon? Which is better?

  Compared to AFINN which assigns a fixed positive score (+1) to "agree", the data-driven approach reveals more nuanced patterns. The word "agree" appears slightly more frequently in negative reviews (4.28e-04) than in positive ones (3.34e-04). This pattern extends to variations like "agrees," which only appears in negative reviews (3.06e-05), and "disagreements," which only appears in positive reviews. The more fine-grained results provided by data-driven approach suggests it:

  1. Learns actual usage patterns from the data instead of relying on predetermined scores
  2. Can capture how different forms of the same word might be used differently
  3. Bases its analysis on relative frequencies rather than absolute sentiment scores.

  Thus, we can conclude that data-driven appraoch is better than sentiment lexicon in this case.